Ch. 12 Notes

Sets

The Set ADT

An unordered collection of **distinct** elements. Core ops:

- `contains(x)` – membership.
- `add(x)` – insert; no-op if already present.
- `remove(x)` – delete; no-op if absent.

Derived: `size`, `empty`, iteration. Everything else (union, intersection, subset, filter, map) is built from these.

Keys vs. elements

The set stores elements x compared by some *key* – often the element itself. When elements are complex (structs), you pass a comparator (ordered set) or a hash + equality (unordered).

Subset testing

$A \subseteq B$ iff $\forall x \in A : x \in B$. Iterate the smaller side, probe the larger. $O(|A|)$ for hashed B, $O(|A| \log |B|)$ for tree-based.

Set operations

$$A \cup B,\ A \cap B,\ A \setminus B.$$

Union / intersection commute; difference does not.

```
// Hashed: iterate one, probe the other
for (auto& x : A)
    if (B.contains(x)) out.insert(x);

// Sorted (std::set): use STL merge-
    style
std::set_intersection(
    A.begin(), A.end(),
    B.begin(), B.end(),
    std::inserter(out, out.end()));
```

Always iterate the smaller set

Intersection cost drops to $O(\min(|A|, |B|))$. Single biggest micro-optimization in set-heavy code.

Cost table

	hashed	**sorted (tree)**
contains	$O(1)^*$	$O(\log n)$
add / remove	$O(1)^*$	$O(\log n)$
union / inters.	$O(\lvert A\rvert + \lvert B\rvert)$	$O(\lvert A\rvert + \lvert B\rvert)$
sorted iter	$O(n \log n)$	$O(n)$
`lower_bound`	–	$O(\log n)$

* expected.

Static vs. dynamic

Static set: fixed membership after construction; supports read-only + new-producing ops. Allows compact encodings (sorted array, bitset, perfect hash, Bloom filter). **Dynamic**: supports `add` / `remove`. Pay more per element for that flexibility.

need	static	dynamic
build once, query	yes	overkill
small memory	yes	harder
mutate at runtime	no	yes
tuned lookup	perfect hash, bitset	hash / tree

Containers you'll actually use

container	order	impl.
`std::set`	sorted	red-black tree
`std::multiset`	sorted, dups	red-black tree
`std::unordered_set`	hash order	open-addr. hash
Python `set`	hash	open-addr. hash
Python `frozenset`	hash, immutable	open-addr. hash
Rust `HashSet` / `BTreeSet`	– / sorted	hash / B-tree

Picking the implementation

- Just need $O(1)$ membership, order doesn't matter: `unordered_set`.
- Need sorted iteration, range queries, or `lower_bound`: `set`.
- Fixed membership, lookup-only: sorted array + `std::binary_search`, bitset, or perfect hash.
- Extremely tight memory, "probably in set" OK: Bloom filter (approximate).

Common patterns

- **Dedup**: `std::set<int> s(v.begin(), v.end())` or `sort + unique + erase`.
- **Seen-set** during traversal: avoid cycles in BFS / DFS.
- **Set as filter**: keep only items whose key is in a whitelist / not in a blacklist.

Top gotchas

- Relying on iteration order in `unordered_set` – implementation-defined and libc-version-dependent.
- Mutating elements in a sorted set in a way that changes their sort key – invalidates the invariant silently.
- Using `set_intersection` / `set_union` from `<algorithm>` on *unsorted* ranges – requires sorted input.
- Writing $A \setminus B$ as $A - B$ in math then forgetting which STL algorithm corresponds (`set_difference`).
- Assuming "const" means truly immutable in C++ – someone can `const_cast` it away.